T-04 · Optimización del motor NER (< 2,0 s por página)

Ajustar el motor local de anonimización (spaCy `es_core_news_md` + reglas) para que la extracción y anonimización de PII no sea un cuello de botella, **sin perder precisión**.

Criterio de aceptación: p95 < 2,0 s por página (página = 3.000 caracteres, ~500 palabras), cobertura de PII = 100 % y precisión ≥ 95 % sobre el corpus etiquetado.

Resultado

Medido con `scripts/benchmark_ner.py` y `tests/test_rendimiento_ner.py` (Python 3.13, spaCy 3.8.16, es_core_news_md 3.8.0, 2 núcleos Xeon 2,8 GHz, sin GPU).

Métrica	Antes (v1.0.0)	Después (v1.1.0)
Página 3.000 caracteres · p50	77 ms	**25 ms**
Página 3.000 caracteres · p95	88–108 ms	**33 ms** (límite: 2.000 ms)
Evidencia completa 8.000 caracteres · p95	219 ms	**87 ms**
Primera anonimización en la API (modelo en frío)	1,9 s bloqueando la API	7 ms (se precarga al iniciar)
Copias del modelo por proceso	2 (~550 MB)	1 (~335 MB)
Precisión / recall (corpus sintético, 60 págs.)	0,857 / 1,0	**0,988 / 1,0**
Precisión / recall (gold `ner_gold.jsonl`)	0,875 / 0,955	**1,0 / 1,0**
Fugas reales encontradas	correos con tilde, "Los Leones 456", cargos ORG	0 en el corpus

El límite de 2,0 s ya se cumplía antes en esta máquina; la optimización deja **~60× de margen** para el servidor de pruebas o un portátil del equipo dentro de Docker, y el trabajo de precisión corrige fugas que la medición dejó al descubierto.

Qué cambió

Rendimiento

1. **Pipeline mínimo** (`ner_sanitizer.cargar_spacy`): el `ner` de es_core_news_md trae su propio tok2vec, así que se carga solo `["ner"]` (fuera tok2vec compartido, morphologizer, attribute_ruler, parser, lemmatizer). El profiling mostró que esos componentes eran ~45 % del tiempo y no aportan nada a las entidades: **mismas entidades byte a byte** (verificado en 130 páginas y en `test_pipeline_minimo_da_las_mismas_entidades_que_el_completo`). Si algún día se cambia a un

modelo cuyo NER dependa de un tok2vec compartido, se detecta al cargar y se usa el pipeline completo automáticamente (log de advertencia).
2. **Segunda pasada acotada**: para nombres EN MAYÚSCULAS el motor reprocesaba el documento completo cada vez que había *cualquier* palabra en mayúscula (ISO, DPO, SAP aparecen en casi toda evidencia) → siempre 2 pasadas. Ahora solo se activa con rachas de ≥ 2 palabras en mayúscula y procesa solo una ventana de ±80 caracteres alrededor de cada una (`nlp.pipe` en lote).
3. **Un solo modelo por proceso**: `seguridad.py` (extractos de eventos de seguridad) cargaba su propia copia completa la primera vez. Ahora recibe la misma instancia (sin modificar `seguridad.py` , que se copia desde el repo ComplianceAI-Prompt-Injection).
4. **Precalentado**: el modelo se carga y ejecuta una vez al iniciar el worker (ya se cargaba) **y la API** (antes la primera petición con un evento de seguridad bloqueaba ~2 s el event loop). En la API no es fatal: sin spaCy sigue funcionando como antes.
5. **No bloquear el event loop**: el gateway ejecuta el NER con `asyncio.to_thread` ; mientras una evidencia se anonimiza, el worker sigue respondiendo (heartbeat de Redis, otras tareas).

Precisión (la "dificultad" de T-04)

6. **Referencias normativas no son direcciones**: "la Ley 21.719", "artículo 14", "ISO 27001" (era el falso positivo de la prueba en vivo del 13-oct y la mayor fuente de falsos positivos del corpus: la precisión de DIRECCION subió de 0,43 a 0,99).
7. **Cargos**: se recortan cargos/tratamientos/verbos de los bordes de un nombre. "Presidenta del Directorio" deja de ser NOMBRE (el otro falso positivo de la prueba en vivo); "Sra. Camila Rojas" → solo "Camila Rojas".
8. **Recall nuevo**: "Gerente General Pedro Soto Rojas" (spaCy lo marca ORG, antes se filtraba el nombre) y "Los Leones 456" (spaCy incluye el número en el LOC, antes no se detectaba).
9. **Correos con tildes**: `matías.díaz@empresa.cl` se anonimizaba solo desde `az@...` , dejando visible `matías.dí` . La regex ahora acepta letras Unicode.

Por qué no ONNX (decisión)

Se evaluó y **no se adopta por ahora**:

- El cuello de botella no era la inferencia del modelo sino trabajo innecesario alrededor de él (componentes que no se usan, una segunda pasada completa, una segunda copia del modelo). Quitando eso, la página queda en ~30 ms: ONNX no cambiaría el resultado frente a 2.000 ms.
- spaCy `es_core_news_md` ya es una arquitectura ligera (CNN en CPU). Los NER en ONNX disponibles para español son transformers (BERT/RoBERTa): más precisos en nombres, pero ~400 MB y más lentos en CPU que la CNN actual; además agregaría `onnxruntime` + tokenizers a la imagen y un modelo que hay que descargar fuera de pip.
- `es_core_news_sm` (más pequeño) se midió: solo ~10 % más rápido y la cobertura de nombres cae a 89 % (fugas). Se mantiene **md**.

Si en el futuro se necesita más precisión en nombres (no velocidad), el lugar para enchufar un NER ONNX es `NERSanitizer(cargar_nlp=...)`, que ya acepta otro cargador.

Limitación conocida (sin cambios respecto de v1.0)

Documentos **enteros en mayúsculas** con frases largas: spaCy puede devolver una entidad MISC gigante y no aislar el nombre. Las rachas de nombre + apellido normales se detectan; para textos 100 % en mayúsculas convendría una regla adicional (pendiente, fuera de T-04).

`seguridad.py` tiene la misma regex de correo ASCII: conviene corregirla en el repo ComplianceAI-Prompt-Injection y volver a copiarla.

Archivos

Tipo	Archivo	Qué hace
modificado	`app/infrastructure/security/ner_sanitizer.py`	v1.1.0: pipeline mínimo, ventanas en mayúscula, modelo compartido, precalentado, filtros de precisión, correo Unicode
modificado	`app/application/ai/gateway.py`	NER en `asyncio.to_thread`
modificado	`app/infrastructure/security/prompt_guard.py`	Comparte el modelo con `seguridad.py`
modificado	`app/main.py`	Precarga del motor NER al iniciar la API (no fatal)
modificado	`tests/data/ner_gold.jsonl`	5 casos nuevos (Ley, cargos, cargo + nombre)
nuevo	`tests/test_rendimiento_ner.py`	18 pruebas: estructurales (siempre) y con el modelo real
nuevo	`tests/_corpus_ner.py`	Corpus sintético etiquetado y determinista
nuevo	`scripts/benchmark_ner.py`	Benchmark p50/p95/p99 + profiling (cProfile)
nuevo	`CAMBIOS_T04.md`	Este documento

No hay migraciones, variables de entorno ni dependencias nuevas.

Cómo probar

```
# Windows, desde Proyecto/backend con el venv activo
pytest tests -v -rxX                          # suite completa (212 pruebas)
pytest tests/test_rendimiento_ner.py -v -s    # solo T-04, imprime p50/p95 y precisión

python scripts/benchmark_ner.py --paginas 100 --evidencia 8000 --perfil
python scripts/benchmark_ner.py --json reporte_t04.json   # evidencia para el informe

# En el servidor de pruebas / Docker (después de reconstruir la imagen)
docker compose up -d --build
docker compose exec worker python scripts/benchmark_ner.py --paginas 100 --evidencia 8000
docker compose exec worker pytest tests/test_rendimiento_ner.py -v -s
```

En producción el tiempo de cada anonimización ya queda registrado en la tabla `sanitizaciones` (`duracion_ms`), así que se puede monitorear sin el script.